# SpendWise — QA and Security

Acceptance tests, edge cases, data privacy and production-readiness checks

## What this document controls

Use after integration. Ask Antigravity to run checks and provide evidence. Tests requiring AWS deployment should be marked separately from local tests.

# COPY THE FULL PROMPT BELOW INTO ANTIGRAVITY

Use this prompt as one message. Let Antigravity inspect the repository, make the requested changes, run checks, and report what it actually completed. Do not skip the checkpoints.

Act as an independent QA engineer and security reviewer. Inspect the entire SpendWise repository and test it; do not simply state that everything works.

```
FUNCTIONAL TESTS
1. Landing page and authentication flow load correctly.
2. User signs in and signs out.
3. Create an expense; it persists after refresh.
4. Create income; dashboard totals update.
5. Edit a transaction and verify only that transaction changes.
6. Delete a transaction and verify it disappears from the backend.
7. Search and filter by date, category and type.
8. Create an overall monthly budget and a category budget.
9. Update and delete budgets.
10. Dashboard income, expense, remaining budget and charts match the saved transactions.
11. Current-versus-previous-month comparison uses the correct date ranges and timezone.
12. Can I Afford This handles zero, empty, negative, very large and valid purchase amounts.
13. Budget warnings occur at 75%, 90% and 100% with sensible boundary behavior.
14. Validate empty description, malformed date, invalid type/category, NaN, Infinity and invalid amount precision.
15. Loading, empty, error and success states are understandable and accessible.

SECURITY TESTS
1. Unauthenticated requests cannot read or change private records.
2. User A cannot read, update or delete User B's transaction or budget by changing IDs.
3. User-supplied userId fields are ignored or rejected; identity comes from verified claims.
4. API responses do not leak internal stack traces or secrets.
5. CORS only permits intended frontend origins.
6. S3 objects are not public; presigned URLs expire and are scoped appropriately.
7. Repository contains no AWS credentials, secrets or real `.env` files.
8. Verify no sensitive data is printed in logs.

ENGINEERING CHECKS
- Run package scripts for lint, type-check, tests and production build.
- Validate SAM template if CLI is installed.
- Inspect console errors, broken links, mobile layouts and keyboard focus.
- Add unit tests for pure calculation helpers and request validation.
- Separate tests that can run locally from tests requiring deployed AWS resources.
- Do not change large areas of code just to satisfy a test without understanding the cause.
- Do not claim tests passed if they did not run.
- Produce a report with PASS / FAIL / NOT RUN, evidence or command output summary, defects fixed, remaining risks and exact r
```